MEDUIM-1：位级操作

模块一

211的二进制表示：11010011

10110110的十进制表示：182

在c/c++中，可以将一个十进制整数&1（例如21&1），得到运算结果，结果为1就是奇数，为0就是偶数。

模块二

一

1.输出结果是：**8 29 21 60**。阐述如下：上述计算均是对每一位进行操作：a&b必须对应位都是一才为一，故结果为00001000，转化为二进制就是八。a|b则是有一为一。a^b是按位异或，相同为零，不同为一。a<<2是将a整体左移两位，右端不灵，b>>1将b右移一位，左端补一，再运算。

2.这个题我们暂且讨论n<32的情况。(根据题目要求，从第一位开始计数，而不是第零位)

```
int f(int x,int n){
    if(n<1||n>32){return 0;}
    return (x>>(n-1))&1;
}
```

3.也是讨论n<32情况

```
int g(int x,int n,int t){
    if(n<1||n>32||(t!=0&&t!=1)){return 0;}
    int mask=1;
    x=x&~(mask<<n-1);
    return x|(t<<(n-1));
}
```

4.

```
//方法一
int h(int x){
    unsigned int t=1;
    int counter=0;
    while(counter<32&&(t&x)!=t){
        counter++;
        t=t<<1;
    }
    if(counter==32){return 0;}
    return t;
}
//方法二
int h(int x) {
    return x & -x;
```

```
}
```

5.目的：比较每一位。

```
int i(unsigned long long a,unsigned long long b){
    while(a<b){
        b=b&(b-1);//清除最低位进行减小，保留相同前缀
    }
    return b;
}
```

思考题

1.逻辑运算与位运算的区别：逻辑运算是对于整体的逻辑关系，位运算是对于每一位的数学关系。联系：逻辑运算和位运算的返回值都由0和1确定。

2.常见性质有

德摩根率：

```
~(a & b) = (~a) | (~b)
~(a | b) = (~a) & (~b)
```

密码学常用：

```
a^a=0 加密：c=a^b，解密：a=c^b;
```

相反数：

```
x=-(~x+1)
```

……

3.补码：w位整数的最高位（w-1）有负的权重,其余权重为正。使得最高位成为符号位的表达形式。

因此可以解释这两种区别：

- 无符号数没有补码表示，因此逻辑右移（左端添零）就是除以2^n
- 有符号数：有补码表示，若是逻辑右移会直接改变符号，而算术右移却是除以二的幂（需要注意：对于负数，算术右移是向下取整，而不是向零取整，因为最右端的+1直接消掉了。）

模块三

```
#include <stdio.h>
int hasCommonChar(const char *s1, const char *s2) {
    int mask1 = 0;
    int mask2 = 0;
    int i = 0;

    while (s1[i] != '\0') {
        mask1 |= (1 << (s1[i] - 'a'));
        i++;
    }

    i = 0;
```

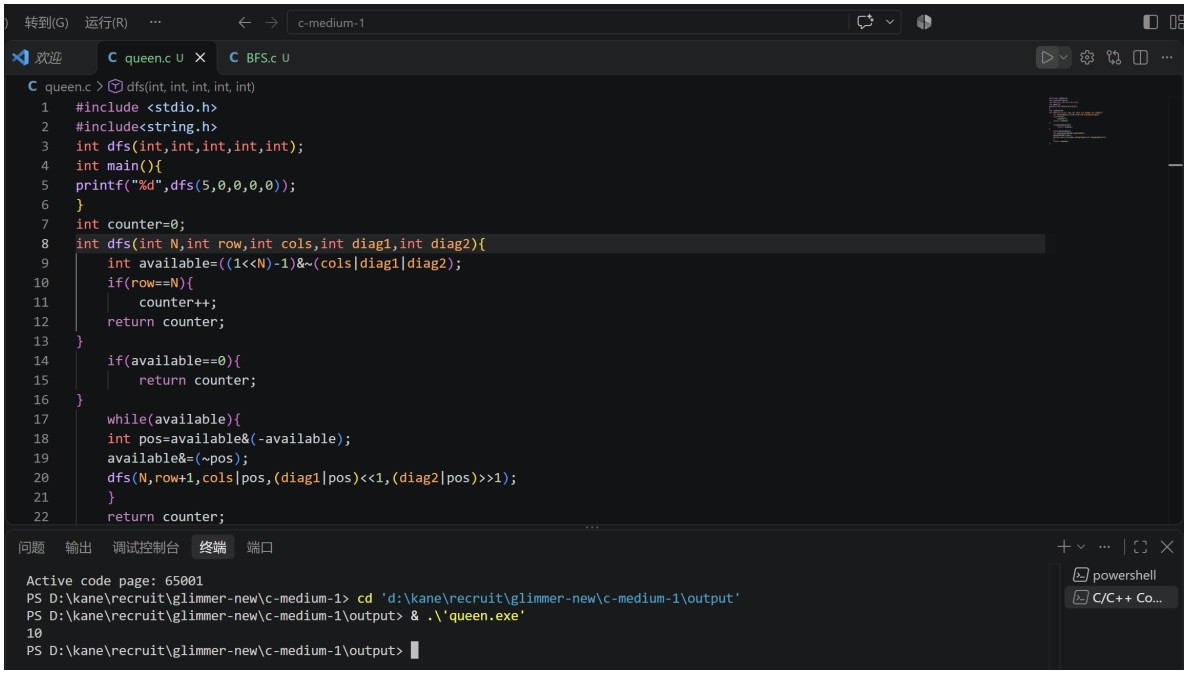
```
    while (s2[i] != '\0') {
        mask2 |= (1 << (s2[i] - 'a'));
        i++;
    }

    return (mask1 & mask2) != 0;
}
```

思考过程：时间复杂度是O(N)，所以不能用双循环遍历，于是用集合存储，又知道'a'等字母排列连续，所以尝试用位运算表示后取交集。

模块四

```
#include <stdio.h>
#include<string.h>
int dfs(int,int,int,int,int);
int main(){
printf("%d",dfs(5,0,0,0,0));
}
int counter=0;
int dfs(int N,int row,int cols,int diag1,int diag2){
  int available=((1<<N)-1)&~(cols|diag1|diag2);
  if(row==N){
    counter++;
  return counter;
}
  if(available==0){
    return counter;
}
  while(available){
  int pos=available&(-available);
  available&=(~pos);
  dfs(N,row+1,cols|pos,(diag1|pos)<<1,(diag2|pos)>>1);
  }
  return counter;
}
```



备注：这是五皇后的运行结果。

模块五

```
#include <stdio.h>
#include <stdbool.h>
// 定义简单队列结构用于 BFS
typedef struct {
    int pos;   // 当前节点编号 (0~15)
    int dist;  // 到达当前节点的最短步数
} Node;
int minStepsToCheese(int walls) {
    int start = 0;
    int target = 15;
    // 如果起点或终点本身是墙，直接不可达
    if ((walls & (1 << start)) || (walls & (1 << target))) {
        return -1;
    }
    // BFS 队列与访问位图
    Node queue[16];
    int front = 0, rear = 0;
    int visited = 0;
    // 起点入队并标记已访问 (请使用位运算)
    queue[rear++] = (Node){start, 0};
    visited |= (1 << start);
    // 上、下、左、右四个方向的节点偏移量
    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};
    //请在TO DO 和END OF TO DO 行之间补全代码:
    while(front < rear)
{
    Node a = queue[front++];
    if(a.pos == target)
        return a.dist;
    int row = a.pos/4;
    int col = a.pos%4;
    for(int i=0;i<4;i++)
    {
        int nr=row+dr[i];
        int nc=col+dc[i];
        if(nr<0 || nr>=4 || nc<0 || nc>=4)
            continue;
        int next = nr*4+nc;
        int test = 1<<next;
        if(walls & test)
            continue;
        if(visited & test)
            continue;
        visited |= test;
        queue[rear++] =
            (Node){next,a.dist+1};
    }
}
    //END OF TO DO
    return -1; // 无法到达
}
int main() {
```

```c
    int walls = (1 << 5) | (1 << 10); // 5号和10号格子是墙
    int steps = minStepsToCheese(walls);
    printf("Minimum steps: %d\n", steps); // 应输出 6
    return 0;
}
```

